

WorkFlow Pro

Test Automation Framework

QA Automation Engineering Assessment Submission

Prepared by: Ritesh Vishwakarma

Role: Senior QA Automation Engineer (Candidate Submission)

Framework Repository:

<https://github.com/Vishwakarma-Ritesh/WorkFlow-Pro>

1. Executive Summary

- Objective: review a flaky login test, design a scalable automation framework, and build an API + UI + mobile integration flow for WorkFlow Pro, a multi tenant B2B SaaS platform.
- Delivered: a working framework using Python, Pytest, Playwright, and BrowserStack, published in the linked repository.
- Framework accomplishes: removes timing based flakiness through condition based waits, separates UI, API, config, and data into independent layers, validates one business flow across API, web, and mobile, and runs in GitHub Actions with Allure reporting.
- Technologies: Python and Pytest for structure and execution, Playwright for auto waiting UI automation, Requests for API checks, BrowserStack for device and browser coverage, Allure for reporting, GitHub Actions for CI/CD.
- Architecture: Page Object Model for UI, a dedicated API client, environment driven config through dotenv, a BrowserStack abstraction, and a test data factory for tenant scoped data.
- Scalability: tenants, browsers, devices, and roles are all configuration driven, not hardcoded, so coverage grows without duplicating test logic.

2. Project Overview

WorkFlow Pro is a B2B project management SaaS platform. Key characteristics:

- B2B SaaS: serves organizational clients on subscription based access.
- Multi Tenant: isolated tenants, for example company1.workflowpro.com and company2.workflowpro.com.
- Multi Role: Admin, Manager, and Employee roles with distinct permissions.
- Cross Browser: expected to work on Chrome, Firefox, and Safari.
- Mobile Enabled: accessible on iOS and Android.
- API Driven: core actions, including project creation, are exposed through REST endpoints.

Testing Challenges

- Tenant isolation must be verified as a security boundary, not just a feature.
- Role based permissions require testing more than one user type per tenant.
- Cross browser and cross device coverage multiplies the execution matrix.
- Data created via API must stay consistent when verified via UI and mobile.

3. Part 1: Flaky Test Audit

3.1 Issues Found

- Hardcoded credentials in the test body.
- No explicit or condition based waits before assertions.
- Exact URL equality assertion, breaks on redirects or query parameters.
- No handling for two factor authentication.
- Tenant isolation checked only positively, absence of other tenants' data not verified.
- No Page Object Model, selectors written directly inside tests.
- No fixtures for browser or page setup and teardown.
- No retry mechanism for transient failures.
- No cleanup of created data or sessions.
- No logging for CI failure diagnosis.
- No browser or viewport parametrization.

3.2 Why CI Differs From Local

- CI runners share resources across parallel jobs and start cold, unlike a warm local machine.
- Network latency and dashboard render time are more variable on CI.

- The test asserts immediately after action instead of waiting for a stable state, so the race is lost more often under CI load.
- Tenant specific data volume can change dashboard load time, making the test tenant dependent as well.

3.3 Fixes Implemented

- Credentials and tenant data sourced from environment variables and a test data factory.
- Locator based auto waiting and expect assertions with timeouts replace fixed delays.
- URL checks use path or pattern matching instead of strict equality.
- Conditional step detects and completes 2FA using a configured test OTP.
- Tenant isolation verified both positively and negatively.
- Page Object classes encapsulate selectors and actions.
- Pytest fixtures manage browser and page lifecycle with guaranteed teardown.
- Retry utility with backoff applied only to known transient conditions.
- API level teardown removes data created during the test.
- Structured logging captures step, tenant, and platform context.
- Browser and viewport parametrized through fixtures and CI matrix.

3.4 Problem, Impact, Solution

Problem	Impact	Solution
Hardcoded credentials	Cannot test multiple tenants or roles	Env variables and test data factory
No explicit waits	Intermittent assertion failures	Locator based auto waiting
Exact URL assertion	Breaks on redirects	Path or pattern based match
No 2FA handling	Fails for 2FA accounts	Conditional 2FA step
Weak isolation check	Cross tenant leakage undetected	Positive and negative isolation checks
No Page Object Model	UI change breaks many tests	Page Object classes
No fixtures	Leaked sessions on failure	Pytest fixture lifecycle
No retry logic	Transient failure fails run	Scoped retry with backoff
No cleanup	Residual test data	API level teardown
No logging	Hard to diagnose CI failures	Structured logging
No browser coverage	Cross browser issues missed	Parametrized execution

4. Part 2: Framework Design

4.1 Repository Structure

```
workflow-pro-automation/
|-- src/
|   |-- pages/          Page Object classes
|   |-- api/            REST API client
|   |-- fixtures/       Pytest fixtures
|   |-- config/         Env and tenant config loaders
|   |-- browserstack/   Capability and session management
|   |-- utils/          Retry, logging, data factory
|-- tests/
|   |-- ui/  api/  mobile/  integration/
|-- config/  .env.example, tenants.yaml
|-- reports/          Allure results and HTML output
|-- docs/
|-- .github/workflows/ CI/CD pipeline
|-- pytest.ini
|-- requirements.txt
```

4.2 Folder Responsibilities

- pages: one class per screen, exposes action methods, not raw selectors.
- api: thin client over Requests, tenant aware headers, typed methods per endpoint.
- fixtures: browser, API session, and test data setup with guaranteed teardown.
- config: loads env variables and tenant definitions into one config object.
- browserstack: builds capability sets, same interface for local or remote sessions.
- utils: retry decorator, structured logger, test data factory.
- tests: split by layer, UI, API, mobile, integration, for targeted execution.
- reports and docs: Allure output and framework documentation.

4.3 Key Design Points

- Page Object Model: isolates UI change impact to one class, reused across UI and integration tests.
- Fixtures: session scoped for config and BrowserStack connection, function scoped for browser context and data, teardown built into fixture lifecycle.
- Configuration: .env locally, GitHub secrets in CI, tenant values in tenants.yaml, no code change needed to target a new environment.
- BrowserStack abstraction: single interface for local or remote sessions, new devices added as configuration.
- API client: tenant aware headers and typed methods, reused for both API tests and integration setup.
- Scalability: every growth dimension, tenants, browsers, devices, roles, is configuration, so suite size scales with business flows, not platforms.

5. Part 3: API + UI + Mobile Integration Flow

5.1 Flow

```
API Create Project
|
API Verification (poll for created status)
|
Web Login (tenant scoped user)
|
Project Verification (UI reflects project)
|
BrowserStack Mobile Session (project visible)
|
Tenant Isolation Check (absent from other tenants)
|
```

5.2 Step Explanation

- API Create Project: POST /api/v1/projects with bearer token and X-Tenant-ID header.
- API Verification: confirm success status, retain project id for later steps and cleanup.
- Web Login: authenticate a UI session for a user in the same tenant.
- Project Verification: confirm the project appears in the UI list using the returned id.
- Mobile Session: confirm the same project is visible on a BrowserStack device session.
- Tenant Isolation Check: confirm the project is absent from a different tenant's view.
- Cleanup: delete the project through the API, scoped to its test run id.

5.3 Key Reasoning

- API before UI: removes UI variability from setup, isolating failures to the layer under test.
- Eventual consistency: UI verification polls within a bounded window instead of asserting instantly.
- Retry strategy: backoff applied only to the known consistency window, never to isolation or security assertions.
- Cleanup strategy: every created record is tagged and torn down through a fixture, regardless of pass or fail.

6. Assumptions

- A bearer token model is used for API authentication, configured per environment.
- A test specific bypass or fixed configurable OTP is available for 2FA enabled test accounts.
- Tenants and test users for Admin, Manager, and Employee roles are pre provisioned in a staging environment.
- Valid BrowserStack credentials are supplied through environment variables and CI secrets.
- Allure and GitHub Actions artifact storage are acceptable as the reporting destination.

7. Missing Requirements

- Production authentication mechanism for the API is not defined.
- No test specific process to bypass or retrieve OTP for 2FA accounts.
- Tenant provisioning process, API or admin driven, is not specified.
- Documented tenant data isolation and retention policy is not provided.
- Permission matrix for Admin, Manager, and Employee roles is not defined.
- Acceptable eventual consistency window between API write and UI visibility is not specified.
- API rate limits for parallel test execution are not defined.
- Official browser and OS version support matrix is not provided.
- Supported mobile devices and OS versions are not specified.
- Clarity on native app versus mobile web scope is missing.
- BrowserStack usage budget and parallel session limits are not defined.
- Ownership and rotation process for CI secrets is not documented.
- Stable staging environment with managed test data is not confirmed.
- Contract testing and visual regression tooling standards are not defined.
- Data privacy requirements for test data generation are not specified.

8. Tooling Decisions

Tool	Reason Selected
Python	Readable, strong ecosystem for UI and API testing

Tool	Reason Selected
Pytest	Fixture model, parametrization, Allure plugin support
Playwright	Auto waiting, one API across browsers, native tracing
Requests	Simple, reliable HTTP client for API checks
BrowserStack	Real device and browser coverage without owning infrastructure
Allure	Structured, historical reporting for engineering and stakeholders
GitHub Actions	Native CI integration, matrix builds, artifact storage
python-dotenv	Keeps environment values out of source control
Retry utilities	Separates genuine defects from known transient conditions
Page Object Model	Centralizes selectors, lowers UI maintenance cost
Test Data Factory	Unique tenant scoped data, avoids collisions

9. Testing Strategy

- Smoke: fast core checks on every push.
- Regression: full core flow coverage on merges to main.
- Integration: combined API, UI, mobile flow, scheduled and pre release.
- API and UI: layer specific contract and behavior checks.
- Mobile: real device checks through BrowserStack.
- Cross Browser: core UI suite parametrized across Chrome, Firefox, Safari.
- Tenant Isolation and Security: role based access and cross tenant absence checks.
- Data Validation: checks on data shape and correctness, not just status codes.
- Parallel Execution: independent, isolated tests run concurrently.
- Prioritization: smoke and high risk flows run first.
- Failure Recovery: retries and guaranteed teardown prevent cascading failures.

10. CI/CD Pipeline

```
Developer Push -> Checkout -> Python Setup
-> Install Dependencies -> Playwright Install
-> Lint -> Execute Tests
-> Generate Reports -> Upload Artifacts
```

- Matrix strategy runs browser, tenant, and role combinations in parallel.
- Dependency and browser binary installs are cached between runs.
- New coverage added through matrix configuration, not pipeline changes.

11. Monitoring and Reporting

- Allure: structured, historical, step level reporting.
- HTML reports: quick local fallback view.
- Logs: structured, tenant and platform aware.
- Playwright trace and screenshots: captured on failure.
- Video recording: enabled for BrowserStack mobile sessions.
- GitHub Artifacts: reports, traces, and media retained per run.
- Failure analysis and metrics: pass rate, duration, and flaky frequency tracked over time.

12. Security Considerations

- No credentials committed to the repository.
- Local execution reads secrets from an untracked .env file.
- CI reads secrets from GitHub Actions encrypted secrets.
- Tokens held in memory only, never written to logs or reports.
- Tenant isolation treated as a security control and verified directly.
- Role based access tested for both allowed and denied actions.

13. Scalability Considerations

- 1000+ tests: growth driven by business flows, not tenant duplication.
- Parallel execution: fixture isolation supports concurrent runs.
- Multiple tenants and additional browsers: added through configuration.
- Cloud execution: BrowserStack abstraction already separates session logic from tests.
- Microservices: API client wrappers extend independently per service.
- Contract, visual, and performance testing: natural extensions of the existing API client and capture layers.

14. Future Enhancements

- Visual regression testing across tenants and browsers.
- Contract testing between API and consumers.
- Automated accessibility checks in the UI suite.
- Performance and load testing reusing the API client.
- Resilient or self healing locator strategies.
- Stakeholder facing dashboard for pass rate and flaky trends.

15. Final Notes

- Framework built strictly from the assessment's stated requirements.
- No live application, production API, BrowserStack credentials, or real users were provided, so the framework is fully environment driven.
- Supplying valid credentials, tenants, and URLs through .env and CI secrets allows execution without any code changes.